

Theta Gainers Algo — Developer Handoff

Audience: the engineer continuing this build. **Read time:** ~25 minutes. Read it once before opening the code. **Last updated:** 2026-04-29

1. What this product is

A web platform that places **NIFTY / SENSEX index-options orders** into one or more brokers from a single screen. Two trader workflows:

1. **Default Strategy** — one-click "sell deep-OTM strangle, $\geq 2.5\%$ OTM, target ₹5,000 premium per ₹1Cr margin used". This is the recommended path for traders who shouldn't be making manual decisions.
2. **Manual builder** — pick strikes / lots / premium-trigger / exit rules and place a custom multi-leg order.

Compliance: SEBI rate cap (≤ 10 orders/sec), NIFTY freeze 1800 / SENSEX 1000 (auto-iceberg + jitter), algo-ID tagging on every order, hash-chained audit log that cannot be edited or deleted.

2. Trader's exact requirements (verbatim, paraphrased only where needed)

These came from the product owner over multiple sessions. Treat each as a spec line. The implementation already covers most of them — flagged below.

2.1 Default strategy

- Sell deep-OTM strangle. CE + PE rounded to **next strike further from spot** (CE rounds up, PE rounds down) on the underlying's strike grid (NIFTY 50, SENSEX 100). Never closer than the rule asks.
- Distance: $\geq 2.5\%$ OTM by default (configurable).
- Target: **₹5,000 premium per ₹1Cr margin used**.
- One click loads the configuration; second click places the order.
- Both Default and any Custom builds **share the same Broker/Demat selection** at the top of the page — applies to all orders on the page.
- Some traders should be **restricted to Default only** (manual builder hidden). Admin-controlled flag.

2.2 Strike selector (auto mode)

- Primary criteria: % **away** / **Points away** / **Delta** — radio choice.
- **Same value for CE & PE OR independent values** — toggle.
- Strikes round **away from spot** (same rule as Default).
- Advanced filters (OI, bid/ask spread, volume, IV rank, regime, time window, etc.) live in a collapsible "Advanced filters" section. CE & PE rules can mirror or be independent.

2.3 Premium trigger (separate module from strike selector)

4 modes:

- **Combined Σ** — fires when CE+PE absolute premium $\geq ₹X$.
- **Per ₹1Cr** — fires when (CE+PE premium $\div$ margin used) $\times 1Cr \geq ₹X$.

- **Per-leg** — each leg has its own threshold (= leg's "Trade Price" field, single source of truth). Fires "linked" (both at once) or "independent" (each fires on its own when met).
- **Enter now** — bypass trigger, place LIMIT immediately.

When trigger is Combined Σ or Per ₹1Cr, the strangle is locked to 2 legs (CE + PE) — Add Leg disables, extras get trimmed.

2.4 Entry time window

- Restrict entries to an intraday IST window (e.g. 09:30–10:30).
- Quick presets (Open / Morn / Mid / Aft / All).
- Toggle to disable restriction entirely.

2.5 Broker & Demat

- Single broker + single demat (default).
- Single broker + **multi-demat** (SOR within broker).
- **Multi-broker + multi-demat** SOR (orders fan across selected broker × demat combos).
- Margin allocation: "Deploy ₹X Cr · keep Y% or ₹Z free per demat". System pulls weighted by available free margin in each demat.
- Cushion = $\max(\% \text{ of demat balance, ₹ floor})$ — kept untouched per demat.

2.6 Exit rules

- Stop Loss (₹), Target (₹), Square-off time (IST), MTM-DD kill (% from peak). All four visible by default.
- Trailing SL (toggle).
- Lock-in profits (toggle).
- **Dead-man switch is in Advanced** — not recommended for Deep OTM intraday; SL/MTM/square-off already cover unattended cases.

2.7 Margin visibility

- Top of page: **Free margin strip** (Total / Used by active strategies / Blocked by pending orders / Free + bar).
- Above action bar: **per-strategy live margin gauge** comparing this trade's required margin to free margin. Red when exceeded; Start / Execute buttons disable. Pre-trade RMS rejects on submit too.

2.8 Confirm modals on every action

- **Load Default Strategy** → confirm with full details (broker, demat, spot, strikes, lots, units, premium target, margin estimate).
- **Load + Execute Now** → same details + typed confirm "EXECUTE".
- **Start (Monitor)** → confirm with all leg details + trigger config + margin status + 2-person approval warning if lots ≥ 5 .
- **Cancel / Save Draft** → small confirm with summary.

2.9 Reporting

- Algo-only reports (split paper vs live).
- Portfolio / strategy / demat-linked.
- Separate **F&O Reporting Tool** is a different product — not part of this app.

2.10 Backtest engine

- Lives in a separate folder (05 – Backtest Engine (separate)/) with its own GitHub repo (ThetaBackTest). Out of scope for this dev brief.

3. Architecture (current state)

3.1 Stack

- **Backend:** FastAPI · SQLAlchemy 2.0 async · Alembic · Postgres 15 · Redis 7 · APScheduler.
- **Frontend:** React 18 + TypeScript · Vite · Tailwind · Zustand · React Query · React Router 6.
- **Observability:** structlog (JSON) · health endpoints.
- **Brokers:** Paper (mock) + Zerodha Kite Connect v3 (live wired). Axis Direct, Monarch, JM Financial Blink — adapter stubs.
- **Storage:** Postgres for everything except market data caching (Redis).

3.2 Top-level folder map

```
01 - Main Code (for Developer)/
├─ backend/
│   └─ app/
│       ├── main.py           ← FastAPI entrypoint, registers routers
│       ├── config.py        ← env loader (BaseSettings)
│       ├── db.py            ← async engine + session
│       ├── auth/            ← /auth – login, TOTP, refresh
│       ├── brokers/         ← base + paper + zerodha + registry
│       ├── strike_selector/ ← /strike-selector – engine + 16 filters
│       ├── strategy/        ← /strategy – CRUD + lifecycle
│       ├── execution/       ← /execution – order_manager, iceberg
│       ├── risk/            ← pre-trade RMS + runtime kill-switches
│       ├── audit/           ← hash-chained immutable log
│       ├── notify/          ← multi-channel notifications
│       ├── analytics/       ← /analytics – Deep OTM scoring
│       ├── data/            ← market data, security master
│       ├── health/          ← /health endpoints
│       └─ admin/            ← admin endpoints (reserved)
├─ alembic/                  ← migrations
└─ pyproject.toml

├─ frontend/
│   └─ src/
│       ├── pages/           ← one .tsx per route
│       ├── components/      ← shared UI
│       ├── api/             ← axios client + react-query hooks + ws
│       ├── stores/          ← zustand stores (auth, theme)
│       ├── App.tsx          ← routes
│       └─ main.tsx          ← root
```

3.3 Frontend routes

Path	Page	Notes
/login	Login	TOTP optional
/connect-broker	ConnectBroker	First-time broker session
/	Dashboard	P&L summary, active strategies

/trade	NewStrategy	The main page. Default + manual + auto
/strategy/:id	StrategyMonitor	Live monitor of one running strategy
/strategy/new	NewStrategy	Same as /trade
/templates	Templates	Saved strategy templates
/analytics	Analytics	Deep OTM strike recommendations
/reports/*	Reports	Live + paper, split by strategy/demat
/settings/*	Settings	Brokers, risk, execution, users
/admin/audit	AuditBrowser	Hash-chain audit viewer
/admin/*	Admin	Other admin tools

3.4 Backend routes registered in `main.py`

/health/*	health checks
/auth/*	login, refresh, TOTP
/data/*	market data, security master
/strategy/*	strategy CRUD + lifecycle
/strike-selector/*	evaluate rules against live chain
/analytics/*	deep OTM scoring + market snapshot

Reserved (commented out in `main.py`): `/admin/*` .

4. Key files to read first

In this order. If you read these 6 files, you understand 80% of the system.

1. `backend/app/main.py` — wiring, lifespan, scheduler.
2. `backend/app/brokers/base.py` — adapter contract every broker honours.
3. `backend/app/strike_selector/filters.py` — the 13 composable filters (after the trim — see §5).
4. `backend/app/execution/order_manager.py` — order state machine, iceberg slicer, idempotency.
5. `frontend/src/pages/NewStrategy.tsx` — the Trade page. **Largest file in the project (1443 lines)**. Refactor candidate — see §6.
6. `frontend/src/components/StrikeSelectorBuilder.tsx` — primary criteria + advanced rule tree. Second-largest file (509 lines).

5. Audit findings

5.1 Frontend

File	Issue	Recommendation
pages/NewStrategy.tsx	1443 lines — does too much (state, broker picker, default	Split into 5 components. See §6

	CTA, margin allocator, legs table, premium trigger, exit rules, modals)	for plan.
components/StrikeSelectorBuilder.tsx	509 lines — primary criteria + advanced rule tree + live preview in one file	Extract <RuleTree> and <LivePreview> into siblings.
pages/Settings.tsx	557 lines	Acceptable but split per-tab in v2 if it grows.
pages/Reports.tsx	328 lines, lots of mock data	Wire to backend in Phase 2.
Empty dirs pages/Settings/, pages/Admin/, components/ui/	Reserved for sub-routes that never landed	Removed in this cleanup.

5.2 Backend coverage (after Phase 2)

Endpoint	Why	Status
GET /broker/list	Connected brokers list	✅ stub
GET /broker/{broker}/demats	Demats per broker	✅ stub
GET /broker/margin/summary	Free margin strip	✅ stub
POST /broker/margin/allocate	Per-demat allocation preview	✅ stub
POST /strategy/preview-margin	Live margin gauge before submit	✅ stub
POST /strategy/{id}/start	Start (Monitor)	✅ live
POST /strategy/{id}/execute-now	Execute Now bypass	✅ stub
GET/PUT /admin/users/{id}/permissions	default_only flag	✅ stub
GET /admin/me/permissions	Self-service UI gate	✅ stub
POST /strike-selector/preview	Live Preview against chain	partial — needs primary criteria block
WS /strategy/{id}/stream	Live monitor updates	partial

Stub semantics: routes exist with the right shape and Pydantic schemas; mock data today; swap with broker.session calls / DB persistence when those land. Frontend hooks won't change — the contract is stable.

5.3 Code quality

- **Mock data in production files.** NewStrategy.tsx has hardcoded spot, balances, margin numbers. Move to backend/api hooks. Phase 2.

- **No error boundary.** A render error in `StrikeSelectorBuilder` blanks the page. Add `<ErrorBoundary>` around lazy routes.
- **Console errors from `/analytics/market` , `/health/heartbeat`** when backend is offline — silent retry is fine, but they pollute dev logs. Add `enabled: !isOffline` to react-query hooks.
- **Auth token has no expiry handling** in `stores/auth.ts` . Add 401 interceptor that triggers refresh.

5.4 Removed in this cleanup

- Empty directories: `pages/Settings/` , `pages/Admin/` , `components/ui/` .
- Unused imports flagged by `tsc --noEmit` — none currently (clean).
- Old "Future Modules" sub-folder is intentionally separate; not touched.

6. Refactor progress (NewStrategy.tsx) — -51%

Started at **1443 lines** (one file did everything). After Phase 1B + 2 + 3:

<code>pages/NewStrategy.tsx</code>	708 lines (orchestration only)	
<code>components/trade/</code>		
├─ <code>BrokerDematPicker.tsx</code>	277 lines	✓
├─ <code>DefaultStrategyCTA.tsx</code>	96 lines	✓
├─ <code>EntryTimeWindow.tsx</code>	73 lines	✓
├─ <code>ExitRules.tsx</code>	96 lines	✓
├─ <code>LegsTable.tsx</code>	330 lines	✓
├─ <code>MarginStatusStrip.tsx</code>	65 lines	✓
├─ <code>PremiumTrigger.tsx</code>	205 lines	✓
├─ <code>shared.tsx</code> (KV2)	14 lines	✓
└─ <code>types.ts</code> (Leg, OptType, ...)	18 lines	✓

Still queued (low priority):

- `ConfirmModals.tsx` (~200 lines) — 5 modal definitions could collapse into a typed dispatcher
- `PreviewSummary.tsx` (~50 lines) — easy when needed

State stays in the page; subcomponents take typed props + callbacks. No behavioural changes from refactors. Each component owns its layout constants (e.g. `EntryTimeWindow`'s `PRESETS` list) and helpers (`Field`, `ToggleRow` inside `ExitRules`; `Section`, `QuoteStat`, `Adjust` inside `LegsTable`).

7. Build & run

7.1 Backend

```
cd backend
python -m venv .venv && source .venv/bin/activate
pip install -e .
alembic upgrade head
uvicorn app.main:app --reload --port 8000
```

Required env vars (see `app/config.py`):

- `DATABASE_URL` — async asyncpg DSN

- REDIS_URL
- JWT_SECRET
- FERNET_KEY — for broker token encryption
- KITE_API_KEY / KITE_API_SECRET — Zerodha
- APP_ENV — dev / staging / prod

7.2 Frontend

```
cd frontend
npm install
npm run dev # http://localhost:5173
```

Vite proxies `/api/*` → `http://localhost:8000` . Frontend SPA routes (e.g. `/strategy/42`) are NOT proxied — see `vite.config.ts` .

7.3 Dev login bypass

If backend is offline, paste in browser console to inject a session:

```
localStorage.setItem("navin-auth", JSON.stringify({state:{token:"dev-token",refreshToken:"dev-refresh",user:{id:1,username:"rohan",role:"ADMIN",totp_enabled:false},brokerSession:{broker:"paper",demat:"PAPER-001",expiresAt:"2099-01-01T00:00:00Z"}},version:0}));
location.href="/trade";
```

Default-only mode (hides manual builder):

```
localStorage.setItem("tg-default-only","1"); location.reload();
// remove: localStorage.removeItem("tg-default-only");
```

8. SEBI / compliance checklist

Already implemented:

-  Algo-ID tagging on every order (broker adapter level).
-  Rate cap ≤ 10 orders/sec via token bucket in `execution/` .
-  Iceberg slicer respects NIFTY 1800 / SENSEX 1000 freeze with 100ms jitter between slices.
-  Hash-chained audit log; PG triggers block UPDATE/DELETE on the `audit_log` table.
-  Two-person approval triggered when lots ≥ 5 (front-end gate; backend enforcement TBC).
-  Pre-trade RMS rejects orders exceeding free margin.

Pending:

-  OTR (Order-to-Trade ratio) monitoring service.
-  Daily SEBI compliance report (XML).
-  Multi-leg margin SPAN+ELM live calculation (currently flat estimate).

9. Where to extend things

You want to...	Touch this
Add a new strike-selector filter	backend/app/strike_selector/filters.py + frontend/src/components/StrikeSelectorBuilder.tsx FILTERS const
Add a new broker	Implement brokers/base.py interface; register in brokers/registry.py; add to ALL_BROKERS in NewStrategy
Add a new exit kill-switch	backend/app/risk/runtime.py (loop) + frontend/src/pages/NewStrategy.tsx Exit Rules section
Add a new trader role / permission	backend/app/auth/service.py + frontend/src/stores/auth.ts user shape
New report tab	frontend/src/pages/Reports.tsx
New page	Route in App.tsx + sidebar entry in Layout.tsx + (optional) command-palette entry

10. What NOT to do

- **Don't rebuild from scratch.** The hard parts (broker adapters, audit chain, iceberg, paper sim, RMS loop) work. A rebuild burns 6–8 weeks and gets you a worse version of the same product.
- **Don't move state out of `NewStrategy.tsx` into a global store** until the refactor in §6 is done. Local state + props is fine for one page.
- **Don't add Tailwind plugins** without checking the existing CSS vars in `index.css` (`--accent`, `--success`, etc.) — design tokens are already in place.
- **Don't change the broker adapter interface** without updating all three implementations (paper, zerodha, base).
- **Don't bypass the audit log** — every state change goes through it.

11. Open questions for product owner

1. Should the Default Strategy CTA also be available in `/strategy/new` or only in `/trade`?
Currently both routes render the same page.
2. Multi-broker SOR: when one broker session goes dark mid-execution, should the engine cancel the remainder and re-route, or pause?
3. Premium trigger "Per ₹1Cr": should the margin in the denominator be *current* required margin or *projected at fill* (which can spike on fast moves)?
4. Time window: should it apply to **trigger evaluation** only or also to **active positions** (i.e. force-exit if outside window)?
5. Margin allocation cushion: should the cushion be released back to "free" pool when the position closes, or stay reserved for the next strategy?

12. Contact / context

- **Product owner:** Rohan Shah · rohan@navingroup.in
- **Repo:** <https://github.com/ATSDashboard/AlgoTrading>
- **Backtest repo:** <https://github.com/ATSDashboard/ThetaBackTest>
- **Original assignment:** see `DEVELOPER_ASSIGNMENT.md` at project root.

End of handoff. If anything is unclear, ask before assuming.